- In a class definition, the current object self and the current class are the same—the class being defined.
- If you have a reference to the class, you can open the class with class_eval (or module_eval).

How can this stuff ever be useful in real life? To show you how you can apply this theory about the current class, let's look at a trick called *Class Instance Variables*.

Class Instance Variables

The Ruby interpreter assumes that all instance variables belong to the current object self. This is also true in a class definition:

class_definitions/class_instance_variables.rb

```
class MyClass
  @my_var = 1
end
```

In a class definition, the role of self belongs to the class itself, so the instance variable @my_var belongs to the class. Don't get confused. Instance variables of the class are different from instance variables of that class's objects, as you can see in the following example:

```
class MyClass
  @my_var = 1
  def self.read; @my_var; end
  def write; @my_var = 2; end
  def read; @my_var; end
end

obj = MyClass.new
obj.read             # => nil
obj.write
obj.read             # => 2
MyClass.read         # => 1
```

The previous code defines two instance variables. Both happen to be named @my_var, but they're defined in different scopes, and they belong to different objects. To see how this works, you have to remember that classes are just objects, and you have to track self through the program. One @my_var is defined with obj as self, so it's an instance variable of the obj object. The other @my_var is defined with MyClass as self, so it's an instance variable of the MyClass object —a *Class Instance Variable*.

Spell: Class Instance Variable

If you come from Java, you may be tempted to think that Class Instance Variables are similar to Java's "static fields." Instead, they're just regular